# Geth 다층 캐싱 아키텍처 분석

## 개념적 배경

### 왜 다층 캐싱이 필요한가?

Ethereum 노드는 다음과 같은 성능 도전에 직면합니다:

1. **디스크 I/O 병목**: 블록체인 데이터는 디스크에 저장되며, 빈번한 접근 시 성능 저하
2. **상태 트리 복잡성**: Merkle Patricia Trie의 깊은 구조로 인한 다중 노드 접근 필요
3. **메모리 제약**: 전체 상태를 메모리에 보관하기에는 크기가 너무 큼
4. **동시성 요구**: 다중 고루틴에서 안전한 캐시 접근 필요

### 다층 캐싱 전략

Geth는 계층적 캐싱 구조를 통해 이러한 문제를 해결합니다:

```
Application Layer

↓

L1: LRU Cache (Hot Data)

↓

L2: FastCache (Warm Data)

↓

L3: Disk Database (Cold Data)
```

# 구현 분석

## 1. LRU 캐시 구현 ( `common/lru/` )

### 핵심 데이터 구조

```
// 동시성 안전한 LRU 캐시
type Cache[K comparable, V any] struct {
cache BasicLRU[K, V]
mu sync.Mutex
}

// 기본 LRU 구현
type BasicLRU[K comparable, V any] struct {
list *list[K] // 이중 연결 리스트
items  map[K]cacheItem[K, V] // 해시맵으로 O(1) 접근
cap  int  // 최대 용량
}

// 캐시 아이템
type cacheItem[K any, V any] struct {
elem *listElem[K] // 리스트 요소 포인터
value V // 실제 값
}
```

## LRU 알고리즘 구현

### 1. 접근 시 최신화 (Get 연산)

```
func (c *BasicLRU[K, V]) Get(key K) (value V, ok bool) {
item, ok := c.items[key]
if !ok {
return value, false
}
// 최근 사용된 항목을 리스트 앞으로 이동
c.list.moveToFront(item.elem)
return item.value, true
}
```

### 2. 용량 초과 시 제거 (Add 연산)

```
func (c *BasicLRU[K, V]) Add3(key K, value V) (ek K, ev V, evicted bool) {
// 기존 키가 있으면 업데이트
if item, ok := c.items[key]; ok {
item.value = value
c.items[key] = item
c.list.moveToFront(item.elem)
return ek, ev, false
}
```

```
var elem *listElem[K]
if c.Len() >= c.cap {
// 용량 초과 시 가장 오래된 항목 제거
elem = c.list.removeLast()
evicted = true
ek = elem.v
ev = c.items[ek].value
delete(c.items, ek)
} else {
elem = new(listElem[K])
}

// 새 항목 추가
elem.v = key
c.items[key] = cacheItem[K, V]{elem, value}
c.list.pushElem(elem)
return ek, ev, evicted
}
```

### 동시성 보장

```
func (c *Cache[K, V]) Get(key K) (value V, ok bool) {
c.mu.Lock()
defer c.mu.Unlock()
```

```
return  c.cache.Get(key)
}
```

모든 공개 메서드는 뮤텍스로 보호되어 동시성 안전성을 보장합니다.

## 2. 트라이 캐시 시스템 ( `triedb/hashdb/` )

### FastCache 활용

```
type Database struct {
diskdb ethdb.Database // 영구 저장소
cleans *fastcache.Cache // GC 친화적 클린 노드 캐시
dirties  map[common.Hash]*cachedNode // 더티 노드 캐시
// ... 기타 필드
}
```

### 캐시 계층 구조

#### 1. Clean Cache (L1)

- FastCache 사용으로 GC 압박 최소화
- 디스크에서 읽은 불변 노드 저장
- 메모리 효율적인 바이트 배열 저장

#### 2. Dirty Cache (L2)

- 수정된 노드들의 임시 저장소
- 참조 카운팅으로 생명주기 관리
- 플러시 리스트로 순서 보장

```
type cachedNode struct {
node []byte  // 인코딩된 노드 데이터
parents  uint32  // 참조 카운트
external  map[common.Hash]struct{} // 외부 자식 노드 집합
flushPrev common.Hash // 플러시 리스트 이전 노드
flushNext common.Hash // 플러시 리스트 다음 노드
}
```

### 캐시 삽입 로직

```
func (db *Database) insert(hash common.Hash, node []byte) {
// 이미 캐시된 노드는 스킵
if  _, ok := db.dirties[hash];  ok {
return
}
// 메트릭 업데이트
memcacheDirtyWriteMeter.Mark(int64(len(node)))
// 새 노드 생성 및 삽입
entry := &cachedNode{
node: node,
external: make(map[common.Hash]struct{}),
}
// 플러시 리스트에 추가
```

```
entry.flushPrev = db.newest
if  db.newest != (common.Hash{}) {
db.dirties[db.newest].flushNext = hash
}
db.dirties[hash] = entry
db.newest = hash
if  db.oldest == (common.Hash{}) {
db.oldest = hash
}
}
```

## 3. 상태 캐시 시스템 ( `core/state/` )

### 코드 캐시 구성

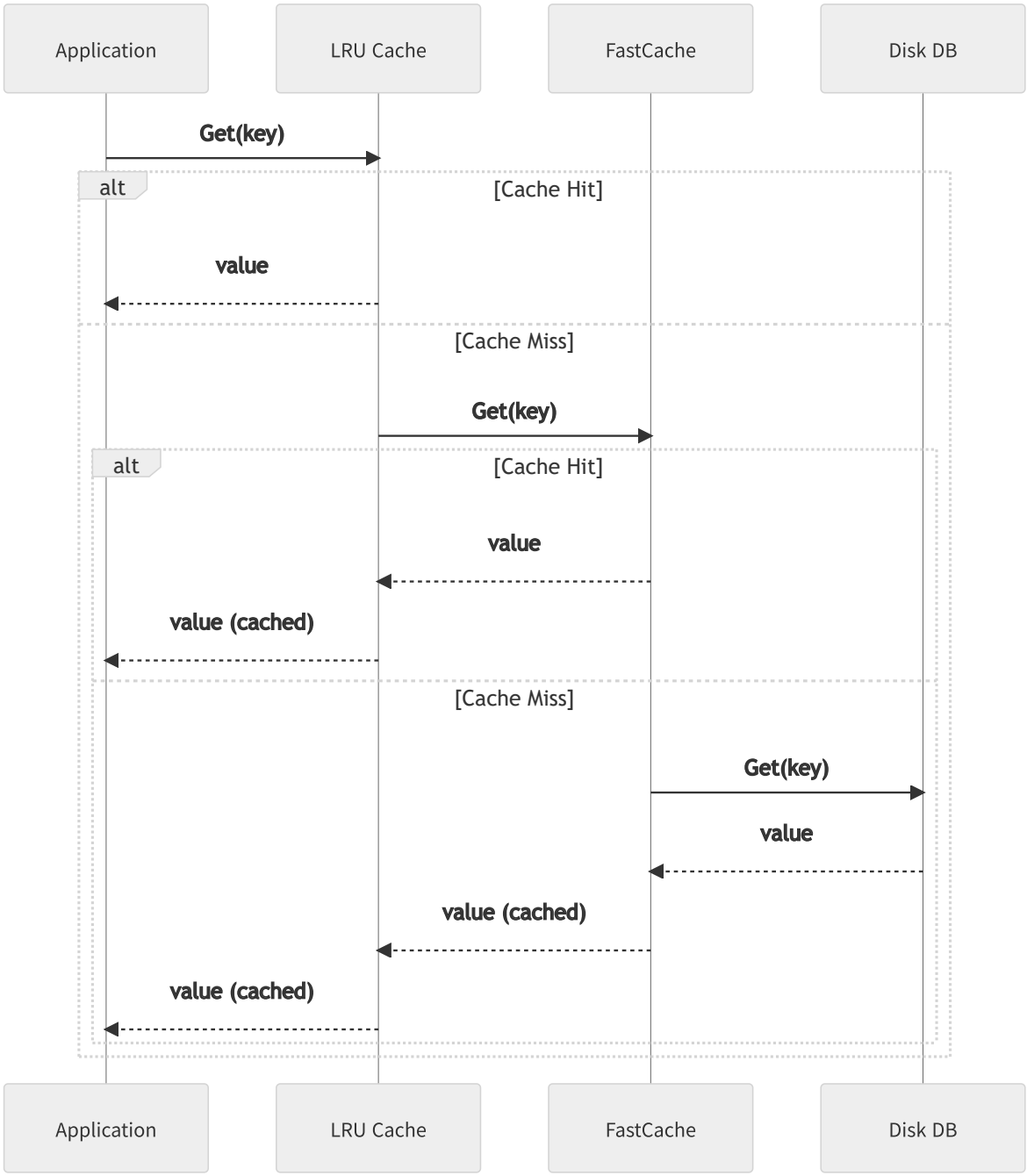
```
const (
// 코드 해시->크기 연관 캐시
codeSizeCacheSize = 1_000_000  // 총 4MB
// 클린 코드 캐시
codeCacheSize = 256 * 1024 * 1024  // 256MB
// 주소->곡선 포인트 연관 캐시
pointCacheSize = 4096
)
```

### 블록체인 캐시 설정

```
const (
bodyCacheLimit = 256  // 블록 바디 캐시
blockCacheLimit = 256  // 블록 캐시
receiptsCacheLimit = 32  // 영수증 캐시
txLookupCacheLimit = 1024  // 트랜잭션 룩업 캐시
)
```

## 실행 흐름

### 캐시 접근 패턴




## 캐시 제거 정책

### 1. LRU 기반 제거

```
// 가장 오래된 항목 제거
func (c *BasicLRU[K, V]) RemoveOldest() (key K, value V, ok bool) {
lastElem := c.list.last()
if lastElem == nil {
return key, value, false
}
key = lastElem.v
item := c.items[key]
delete(c.items, key)
c.list.remove(lastElem)
return key, item.value, true
}
```

### 2. 참조 카운팅 기반 제거

```
func (db *Database) dereference(child common.Hash, parent common.Hash) {
node := db.dirties[child]
if node == nil {
return
}
// 참조 카운트 감소
if node.parents > 0 {
node.parents--
```

```
}

// 참조가 없으면 제거 대상

if  node.parents == 0 {

// 가비지 컬렉션 대상으로 표시

delete(db.dirties, child)

}

}
```

# 성능 최적화

## 1. 메모리 사용량 최적화

### FastCache 사용 이유

- Go GC의 영향을 받지 않는 오프힙 메모리 사용
- 대용량 캐시에서 GC 일시정지 시간 최소화
- 메모리 단편화 방지

### 메모리 풀링

```
// 리스트 요소 재사용

if  c.Len() >= c.cap {

elem = c.list.removeLast() // 기존 요소 재사용

evicted = true

// ... 정리 작업

} else {
```

```
elem = new(listElem[K]) // 새 요소 생성

}
```

## 2. 캐시 히트율 최적화

### 지역성 활용

- 최근 접근된 데이터를 리스트 앞쪽에 배치
- 공간적 지역성을 위한 관련 노드 그룹핑

### 적응적 캐시 크기

```
type BlockChainConfig struct {

TrieCleanLimit  int  // 클린 트라이 노드 캐시 (MB)

TrieDirtyLimit  int  // 더티 트라이 노드 캐시 (MB)

SnapshotLimit  int  // 스냅샷 캐시 (MB)

}
```

## 3. 동시성 최적화

### 읽기 전용 연산 최적화

```
func (c *Cache[K, V]) Peek(key K) (value V, ok  bool) {

c.mu.Lock()

defer  c.mu.Unlock()

return  c.cache.Peek(key) // LRU 순서 변경 없이 조회

}
```

### 배치 연산 지원

```
func (c *Cache[K, V]) Keys() []K {
c.mu.Lock()
defer c.mu.Unlock()
return c.cache.Keys() // 모든 키를 한 번에 반환
}
```

## 실제 사용 예시

### 1. 트라이 노드 캐싱

```
// 트라이 노드 읽기
func (db *Database) node(hash common.Hash) ([]byte, error) {
// 1. 더티 캐시 확인
if node := db.dirties[hash]; node != nil {
return node.node, nil
}
// 2. 클린 캐시 확인
if db.cleans != nil {
if enc := db.cleans.Get(nil, hash[:]); enc != nil {
memcacheCleanHitMeter.Mark(1)
return enc, nil
}
```

```
}
// 3. 디스크에서 읽기
enc := rawdb.ReadTrieNode(db.diskdb, hash)
if len(enc) != 0 {
if db.cleans != nil {
db.cleans.Set(hash[:], enc)
memcacheCleanWriteMeter.Mark(int64(len(enc)))
}
}
return enc, nil
}
```

## 2. 블록 캐싱 설정

```
func NewBlockChain(db ethdb.Database, config *BlockChainConfig) (*BlockChain, e
// 캐시 크기 설정
bodyCache := lru.NewCache[common.Hash, *types.Body](bodyCacheLimit)
blockCache := lru.NewCache[common.Hash, *types.Block](blockCacheLimit)
receiptsCache := lru.NewCache[common.Hash, []*types.Receipt](receiptsCacheLimit)
bc := &BlockChain{
bodyCache: bodyCache,
blockCache: blockCache,
receiptsCache: receiptsCache,
```

```
// ... 기타 초기화
}
return bc, nil
}
```

### 3. 캐시 성능 모니터링

```
// 캐시 히트/미스 메트릭
var (
memcacheCleanHitMeter = metrics.NewRegisteredMeter("hashdb/memcache/clean/hit",
memcacheCleanMissMeter = metrics.NewRegisteredMeter("hashdb/memcache/clean/miss"
memcacheDirtyHitMeter = metrics.NewRegisteredMeter("hashdb/memcache/dirty/hit",
memcacheDirtyMissMeter = metrics.NewRegisteredMeter("hashdb/memcache/dirty/miss"
)

// 사용 예시
func (db *Database) node(hash common.Hash) ([]byte, error) {
if node := db.dirties[hash]; node != nil {
memcacheDirtyHitMeter.Mark(1)
return node.node, nil
}
memcacheDirtyMissMeter.Mark(1)
// ... 다음 레벨 캐시 확인
```

```
}
```

# 관련 시스템

## 의존성 관계

1. **상태 관리 시스템**: 계정 및 스토리지 상태 캐싱
2. **블록체인 동기화**: 블록 및 헤더 캐싱
3. **트랜잭션 풀**: 트랜잭션 검증 결과 캐싱
4. **EVM 실행**: 코드 및 실행 결과 캐싱

## 상호작용 패턴

- **읽기 집약적 워크로드**: 캐시 히트율 극대화
- **쓰기 집약적 워크로드**: 더티 캐시 효율적 관리
- **메모리 압박 상황**: 적응적 캐시 크기 조정

## 영향 범위

캐시 성능은 다음 영역에 직접적 영향을 미칩니다:

- **블록 검증 속도**: 상태 트리 접근 최적화
- **동기화 성능**: 블록 데이터 캐싱으로 네트워크 부하 감소
- **RPC 응답 시간**: 자주 요청되는 데이터의 빠른 응답
- **메모리 사용량**: 효율적인 캐시 관리로 메모리 최적화

이러한 다층 캐싱 아키텍처를 통해 Geth는 디스크 I/O를 최소화하고, 메모리를 효율적으로 사용하며, 높은 성능을 달성합니다.